Interfaces en Programación Orientada a Objetos

Contratos, Abstracción y Polimorfismo

Diseño de Software

2 de octubre de 2026

¿Qué es una Interfaz?

Una interfaz es un **contrato estricto** que define qué debe hacer una clase, pero no cómo debe hacerlo.

- **Abstracción pura:** En su forma clásica, solo contiene firmas de métodos vacíos y constantes.
- **Desacoplamiento:** Separa la definición del comportamiento de su implementación concreta.
- **Multitipo:** Permite que objetos de clases completamente distintas sean tratados bajo el mismo tipo si implementan la misma interfaz.

Nota Técnica

A diferencia de la herencia de clases (relación "Es un"), las interfaces definen capacidades (relación "Puede hacer" o "Se comporta como").

¿Para qué funciona y cómo se utiliza?

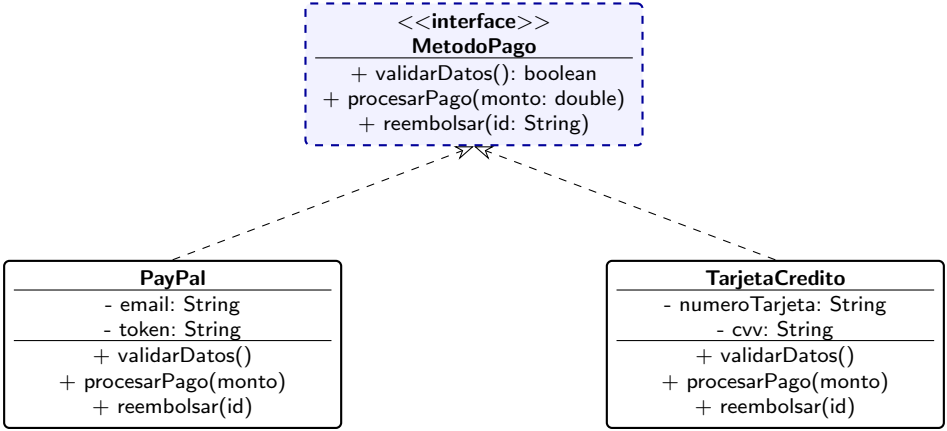
Funciones principales:

- **Simular herencia múltiple:** Java, C# y otros lenguajes no permiten heredar de múltiples clases, pero sí implementar múltiples interfaces.
- **Polimorfismo avanzado:** Permite inyectar dependencias e intercambiar implementaciones en tiempo de ejecución sin alterar el código cliente.

Reglas de uso:

1. Se declara con la palabra reservada `interface`.
2. Una clase la adopta usando la palabra `implements`.
3. La clase **debe** sobrescribir (implementar) absolutamente todos los métodos definidos en la interfaz, o declararse como clase abstracta.

Ejemplo Arquitectónico: Pasarela de Pagos



Implementación: La Interfaz (El Contrato)

MetodoPago.java

```java
public interface MetodoPago {

  // Constante (impl citamente public static final)
  int TIEMPO_ESPERA_MAX_SEGUNDOS = 30;

  // Los m todos son public abstract por defecto
  // Cualquier clase que implemente esto, DEBE tener estos m todos
  boolean validarDatos();

  void procesarPago(double monto);

  void reembolsar(String idTransaccion);
}
```

Implementación 1: Clase PayPal

PayPal.java

```
1  public class PayPal implements MetodoPago {
2    private String email;
3
4    public PayPal(String email) {
5      this.email = email;
6    }
7
8    @Override
9    public boolean validarDatos() {
10     return email.contains(''@'');
11   }
12
13   @Override
14   public void procesarPago(double monto) {
15     if (validarDatos()) {
16       System.out.println(''Cobrando $'' + monto + '' via PayPal a: '' +
```

Implementación 2: Clase Tarjeta de Crédito

TarjetaCredito.java

```java
public class TarjetaCredito implements MetodoPago {
  private String numeroTarjeta;

  public TarjetaCredito(String numeroTarjeta) {
    this.numeroTarjeta = numeroTarjeta;
  }

  @Override
  public boolean validarDatos() {
    return numeroTarjeta.length() == 16;
  }

  @Override
  public void procesarPago(double monto) {
    if (validarDatos()) {
      // Muestra solo los  ltimos  4 d gitos
```

Polimorfismo en Acción: El Sistema Principal

SistemaPagos.java

```
1  public class SistemaPagos {
2
3    // ATENCI N: El par metro es la INTERFAZ, no la clase concreta.
4    //  Este  m todo acepta PayPal, Tarjetas o cualquier m todo futuro!
5    public static void ejecutarCobro(MetodoPago metodo, double total) {
6      System.out.println(''Iniciando transacci n...'');
7      metodo.procesarPago(total);
8    }
9
10   public static void main(String[] args) {
11     // Instanciamos objetos de diferentes clases
12     MetodoPago pago1 = new PayPal(''usuario@mail.com'');
13     MetodoPago pago2 = new TarjetaCredito(''1234567890123456'');
14
15     // El mismo m todo reacciona distinto seg n el objeto que reciba
16     ejecutarCobro(pago1, 500.0);
```

Clase Abstracta vs. Interfaz

Característica	Interfaz	Clase Abstracta
Herencia	Una clase puede implementar **múltiples** interfaces.	Una clase solo puede heredar de **una** clase abstracta.
Estado (Atributos)	Solo puede tener constantes (`public static final`).	Puede tener variables de instancia (estado) de cualquier tipo.
Constructores	No tienen constructores.	Tienen constructores (llamados por las subclases).
Propósito	Definir un rol o capacidad transversal.	Proveer una base jerárquica con código reutilizable.

Ejercicio Práctico

Contexto: Estás diseñando un sistema para un zoológico.

Instrucciones:

1. Crea una interfaz llamada `Nadador` con un método `nadar()`.
2. Crea otra interfaz llamada `Volador` con un método `volar()`.
3. Crea una clase base abstracta `Animal` con un atributo `nombre` y un constructor.
4. **Reto:** Crea la clase `Pato`. Haz que herede de `Animal` e implemente **ambas** interfaces. Codifica la sobrescritura (*@Override*) de todos los métodos requeridos y prueba tu código en un método `main`.